

AsoulAI ChronosFix

A-CFX：带证据的软件修复引擎

用多 Agent 把事故变成可验证补丁与可复用质量资产

官方模板 · 复赛验证版

队伍：AsoulAI

复赛 V1.0



把“会修复”升级为“带证据、可发布、可复用”

复赛版补齐 AgentTeams、云 Skill、评测、失败处理、权限与开源规范。

项目名称

AsoulAI ChronosFix (A-CFX)
带证据的软件修复引擎

问题与场景

多源变更下，AI 能给建议，却难证明因果、验证补丁并让组织放心合并。

核心方案

8 个职能 Worker 协同重放事故；10 个 Skill 能力完成因果、验证、门禁与资产沉淀。

创新差异

Proof-Carrying Change + 可辨识性仲裁：无法区分就拒答，证据不全就阻断发布。

开放复用

开放代码、Schema、12 场景、证据护照、AgentTeams v1beta1 资源与评测脚本。

当前进展

66 项测试；9/9 支持范围诊断正确；1/1 冲突安全拒答；2 项边界透明保留。

复赛不是加功能列表，而是让每个结论都可核验

围绕完整链路、运行证据、失败处理、权限审批与开放规范升级底层工程。

初赛已交付

场景闭环、7 职能 Agent、9 Skill、
确定性 Demo 与公开仓库。

底层逻辑升级

Fail-closed RiskGate、真实 checks、
回滚验证、可辨识性仲裁与哈希绑定。

复赛新增证据

v1beta1 资源、官方 SLS Skill 契约、
12 场景评测、CI、SBOM 与 Demo
评委模式。

输入 Schema

因果仲裁

补丁竞赛

RiskGate

证据护照

一条 run_id 串联 scenario → experiments → checks → PR → run-manifest。

真正昂贵的不是找答案，而是无法放心合并答案

目标用户：中大型研发组织、平台工程团队、云厂商 DevOps / AIOps 产品线。

事故成本

代码、配置、依赖与流量同窗变化，
人工排查慢且知识难沉淀。

组织风险

AI 建议若缺因果、验证、回滚和审
批证据，就无法进入发布流程。

购买理由

让每个 PR 自动携带可审查证据，并
把事故沉淀为跨团队质量资产。

48.72%

Demo 基线失败率

6.25%

补丁最坏失败率

8

强制故障变体

14

证据护照声明

40

自动化测试

商业命题：不是替代研发判断，而是降低评审证据准备、误合并风险与合规审计成本。

从事故信号到可合并 PR，一条 run_id 贯穿闭环

AgentTeams 负责编排，Skill 负责动作，RiskGate 决定是否允许发布。

任务触发

Issue、日志、Trace、Git、配置与流量进入统一场景契约。

协同执行

8 Worker 围绕 IncidentState 分工；
冲突由反事实与可辨识性仲裁裁决。

发布输出

生成补丁变更、回滚契约、checks、PR 草案、证据护照与可复用 Skill 候选。

证据融合

Counterfactual

Fault Genome

补丁竞赛

门禁护照

安全分支：证据不足、check 失败、回滚缺失或无命名审批人 → draft / blocked。

同一个事故，输入是场景契约，输出是变更契约

样例：订单创建接口在 120 RPS 下出现 48.72% 失败率与 606.96 ms P99。

输入 Schema

scenario.json：事件、基线、假设、补丁候选、变体与 Ground Truth。

因果结论

连接池缩容是主因；依赖延迟是放大器；日志代码变更非因果。

变更契约

4 个候选补丁在同一故障族上竞争；胜者 P-RESTORE-POOL 进入 RiskGate。

发布状态

quality_gate=passed；
release=approved；审批人和理由进入摘要哈希。

- 18 个实测 Span、3 个必须 checks、20 项发布工件；全部关联同一 run_id。
- Run Manifest 绑定场景、补丁、回滚、审批和产物 SHA-256。
- 本页数据来自合成确定性回放，不代表真实生产准确率或 MTTR。

第二章

AgentTeams 多 Agent 协同

必选基座：编排、拆解、上下文、执行、追踪

多 Agent 协同与自主闭环能力 25%

AgentTeams v1beta1：正式资源已离线校验

证据驱动 DAG；4 个候选在同一故障族上竞争，revision 共享状态。



角色靠状态契约协同，而不是在聊天室投票

每个 Worker 只写负责的状态分区；Commander 只做路由、裁决与升级。



先证明“能区分”，再决定“能发布”

可辨识性仲裁与 RiskGate 分别解决“结论可靠”和“动作可放行”。

同干预冲突

多个来源假设若映射到相同干预，统一降级为 indeterminate。

无主因证据

没有可辨识 primary cause 时，counterfactual check 失败并阻断 PR 放行。

验证不完整

强制变体、Evidence Passport、回滚或 required checks 任一缺失即 fail closed。

中高风险审批

必须提供命名审批人和理由；布尔 approve 不能替代人工身份。

- quality_gate 与 release_decision 分离：技术通过不等于已获发布授权。
- approval digest 绑定 scenario hash、patch hash、patch id 与 policy version。
- 证据不足时给出 blocker，而不是为了 Demo 生成绿色成功状态。

第三章

Skill 工程与云操作

核心 Skill、官方云 Skill、鉴权、失败处理与生命周期

Skill 工程体系与生态复用 25%

10 个 Skill 能力覆盖证据、因果、验证与资产复用

每项能力都有稳定契约、权限边界、验证方式和版本演进路径。

EvidenceFusion

多源证据与缺口

ChangeTimeline

时序对齐与去重

CounterfactualReplay

干预效果回放

Identifiability

同干预冲突仲裁

FaultGenome

故障变体生成

PatchTournament

补丁竞赛评分

RiskGate

三态门禁审批

EvidencePassport

证明与缺口声明

Report + SkillForge

报告与复用候选

生命周期：候选 → 回放评测 → 人工评审 → 版本发布 → 监控 → 回滚；新 Skill 不自动上线。

官方 SLS Skill 已进入 Worker 工具契约

默认 dry-run；真实查询需要参赛者在本地配置只读 RAM Profile。

官方来源

skills.aliyun.com
alibabacloud-sls-query
上游 commit 已锁定。

鉴权边界

RAM Profile / 环境凭证
密钥不落盘

Worker 编排

18 个实测 Span；
`patch\_tournament\_completed` 将 4 个候选排名写入
Matrix 事件流。

失败处理

先做 Schema 校验
CLI / 权限错误入审计
返回 evidence missing

- 最小 RAM 权限：log:GetIndex、log:GetLogStoreLogs；User-Agent 标注 Skill 来源。
- 已完成 adapter 契约测试与 dry-run 证据；没有声称成功访问真实 SLS。
- 真实端到端验证需要 Project、Logstore 和只读 RAM Profile，且不会要求在聊天中粘贴密钥。

不堆产品：先有证据闭环，再平滑迁移云上

每个推荐组件都说明必要性、权限边界、可替换性与迁移成本。

当前已实现

v1beta1 资源、本地内核、SLS 只读适配器、JSON 证据与 GitHub 草案。

已验证契约

Schema、鉴权、错误、审批、回滚、审计均有文档和测试。

生产化目标

AgentTeams / Nacos / Higress
PolarDB / RocketMQ / 可观测

能力域	当前可验证实现	目标组件
协同 / 治理	v1beta1 声明 + 本地编排	AgentTeams + Nacos + Higress
数据 / 事件	JSON / JSONL + Run Manifest	PolarDB + UnifiedModel + RocketMQ
可观测 / 评测	Trace / Log / Metrics + 12 场景	LoongSuite + Studio / AgentLoop

第四章

工程验证、安全与可审计

对应官方 PPT 框架与评分维度

工程落地与安全可审计 20%

一次运行不是截图，而是一组互相绑定的证据

scenario、patch、rollback、approval 与 20 项发布工件由 run-manifest 哈希绑定。

Trace

18 个 Span；起止时间、duration 与 parent span 全部实测。

Checks

反事实、故障变体、回滚
契约 3 项 required checks
全部执行并记录 exit code

RiskGate

quality_gate=passed；
release=approved；命名审
批人、理由与策略版本可
追溯。

Run Manifest

场景、变更、回滚、审批
输入和每个证据文件均记
录 SHA-256。

- Metrics 明确区分 measured / derived；外部工具未调用时 success_rate=null。
- PR 草案直接读取选中 changes、rollback_changes、checks 与 gate，不再硬编码成功。
- 当前 release_id：acfx-5be047a48f03；run_id / trace_id 可在 evidence/ 复核。

评测不藏失败：9/9 正确诊断，1/1 安全拒答

12 个合成场景：9 Golden、2 Badcase、1 Insufficient Evidence；不代表生产准确率。

Golden

当前确定性模拟器支持范围内 9/9 命中 Ground Truth。

冲突拒答

相同干预对应多个来源假设时 1/1 安全 abstain。

已知漏诊

代码版本与队列积压尚未建模，2 项 Badcase 不计入成功。

12

总样例

10/12

达成预期

100%

受支持诊断

100%

冲突拒答

0

意外强行归因

口径：9/9 与 1/1 是合成确定性回放结果；两项 unsupported 漏诊透明保留。

Badcase 公开保留，边界才能变成路线图

不受支持的根因不强行归因；可辨识性不足时明确 abstain。

代码回归

code_version 只作哨兵，尚未进入行为模型；当前漏诊并阻断发布。

队列积压

队列深度与消费速率尚未建模；不冒充 RocketMQ 根因回放。

来源冲突

两个来源假设映射到同一干预时标记 indeterminate，等待来源级证据。

样例	系统行为	发布结果
code-regression	abstain / 已知漏诊	blocked
queue-backlog	abstain / 已知漏诊	blocked
conflicting-sources	indeterminate	待来源级证据

评委可以按 README 从零复现核心闭环

依赖、配置、样例输入输出、资源声明和运行证据全部进入仓库。

核心入口

```
demo.py --approve  
--approver <name>  
--approval-reason <reason>
```

AgentTeams 入口

agentteams/run_chronosfix
_team.py 输出 compatible
transcript 与同源证据。

契约与 CI

18 个实测 Span; 20 项发
布工件与源码锚点由
release-manifest 统一校验

Demo 交付

GitHub Pages 静态站读取生
成的 demo-data.json; 不保
存凭证, 不写生产。

- CI 同时运行 12 场景评测、严格 JSON/JSONL 校验与 AgentTeams 资源校验。
- 核心运行仅依赖 Python 标准库; 可选 PyYAML 用于资源声明验证。
- 本地执行、dry-run 和真实外部执行在 UI、日志与文档中使用不同状态标签。

商业化卖的不是改代码，而是变更治理

以 DevOps / AIOps 增值模块切入，逐步形成组织级质量资产网络。



- 采购角色：平台工程、研发效能、SRE 与云厂商；收费可采用订阅 + 集成 + 私有部署。
- 价值指标：评审耗时、误合并率、回滚恢复时长、审计准备成本；真实收益需通过试点测量。

开放核心协议与评测，生产执行保持最小权限

开源让方法可复核，权限边界让企业敢接入。



- 当前状态：Controller / Matrix 尚未执行；真实 SLS 需要本地只读凭证；其余推荐组件为迁移目标。
- 原则：组件数量不计分；只在闭环证据需要时引入，并披露替换原因与迁移成本。

AsoulAI 交付完整复赛包，不只是一个概念 PPT

方案、代码、Demo、评测、Trace、审计和开放计划均可独立进入与复核。

队伍名称

AsoulAI

作品名称

AsoulAI ChronosFix：带证据的软件修复引擎

Demo

<https://asoul-official.github.io/asoulai-chronosfix/>

代码仓库

github.com/ASOUL-Official/asoulai-chronosfix

方案材料

[submission/ChronosFix_复赛方案.pptx](#) /
[.pdf](#)

可验证证据

[evidence/](#) + [evaluation-corpus/](#) + [run-manifest.json](#)

核心结论

根因可证明 | 补丁可验证 | 发布可审计
| 经验可复用

A-CFX：让每次软件修复都携带证据，让每次事故都沉淀为质量资产。